

# FastAPI Support/Ops Agent - Flagship Capstone

AI Engineering Career Transition Roadmap - Phases 1-8, unified

Repo: <https://github.com/Bhogeshwarj/ai-learn> (branch: main)

---

## 1. Where this came from

This capstone follows a personal 8-phase AI engineering roadmap built to support a career transition into AI engineering (job/freelance track). Phases 0-6 were completed as separate topic notebooks. Rather than doing Phase 7 (Production) and Phase 8 (Portfolio) as more isolated tutorials, the decision was made to build one flagship project that ties everything together into something real enough to demo live in an interview or client call.

## 2. Phases already completed (0-6)

- **Phase 1 - LangChain fundamentals:** chat models, message roles, tool calling, structured output, middleware.
- **Phase 2 - RAG:** document loading/chunking, embeddings/vector stores, hybrid search, vectorless/PageIndex-style retrieval, agentic RAG.
- **Phase 3 - Evaluation & guardrails:** golden Q/A eval sets, LLM-as-judge, PII/refusal guardrails, prompt-injection awareness.
- **Phase 4 - LLM gateways:** provider fallback, rate limiting, cost tracking, caching.
- **Phase 5 - Agents & LangGraph:** graph-based state machines, human-in-the-loop checkpoints, debugging traces, multimodal agents, MCP.
- **Phase 6 - Deep agents:** long-horizon planning, sub-agent delegation, persistent-memory/file-system-style agents.

These live in the repo under: **Langchain-V1-Crash-Course-main/**, **RAG-Tutorials/**, **LLM- Eval/**, **LLM -Guardrail/**, **LLM - Gateways/**, **Agentic- Langgraph/**, and **Deep Agents/**.

## 3. The capstone project

A support/ops agent for a real open-source project, combining a documentation Q&A layer with genuine support/ops-style tooling - not just retrieval.

**Target project: FastAPI.** Chosen over Supabase, LangChain itself, and Docker because its public docs are a manageable size for hybrid-search chunking, it has a real versioned changelog with genuine deprecations (Pydantic v1 support, **on\_event** startup/shutdown handlers superseded by lifespan), an active GitHub issue tracker, and is recognizable to any technical interviewer - not just AI-specific roles.

## 4. What the system will include

- **RAG core:** loading/chunking FastAPI's public docs, hybrid (keyword + vector) search, citations back to source chunks.
- **Support/ops tools beyond retrieval:** changelog/version lookup, "is X deprecated" checks, GitHub issue search.
- **LangGraph multi-agent loop** with at least one human-in-the-loop approval checkpoint.
- **Deep-agent-style planning:** sub-agent delegation for multi-step questions needing more than one retrieval/tool call.
- **Eval harness:** golden Q&A set, LLM-as-judge scoring, regression tracking across changes.
- **Guardrails:** blocking off-topic/unsafe queries, handling prompt injection carried in retrieved GitHub issue text.
- **LLM gateway layer:** provider fallback and cost tracking in front of raw model calls.
- **Production (Phase 7):** Dockerized, deployed behind a simple API (Render or Fly.io), LangSmith tracing, basic latency/cost/error-rate monitoring, least-privilege tool design, kill switch.
- **Portfolio (Phase 8):** a real README (what/why/how to run/eval results) and a short write-up (problem/approach/trade-offs/results).

## 5. Milestone roadmap

**M0 - Discovery (done).** Confirmed stack versions (langchain $\geq$ 0.3.25, langgraph $\geq$ 0.4.8, langsmith $\geq$ 0.3.45)

as of the most recent existing project; to be re-verified against current docs when M1 starts), picked FastAPI as the target project, decided the gateway should be built right after architecture instead of after eval+guardrails so nothing needs retrofitting. The whole roadmap directory was turned into a git repo and pushed to [github.com/Bhogeshwarj/ai-learn](https://github.com/Bhogeshwarj/ai-learn).

**M1 - Architecture & data acquisition design.** System diagram (LangGraph graph shape; where gateway/eval/guardrails sit), repo layout for a new **fastapi-support-agent/** project, and concretely how to pull FastAPI's docs (markdown source in the [tiangolo/fastapi](https://github.com/tiangolo/fastapi) repo), changelog (release-notes.md), and GitHub issues (REST/GraphQL API, read-only token). Fresh uv-managed project with pinned, currently-verified dependency versions and a chosen vector store (Chroma or FAISS).

**M2 - LLM gateway layer.** Provider fallback, cost tracking, rate limiting, caching in front of raw model calls, built first so every later call routes through it natively. Reuses patterns from the existing LLM - Gateways notebook, updated to current SDK behavior.

**M3 - RAG core.** Load/chunk the FastAPI docs, hybrid (keyword + vector) search, citations back to source doc chunks/URLs.

**M4 - Support/ops tools beyond retrieval.** Changelog/version lookup tool, "is X deprecated" checker, GitHub issue search tool - the layer that makes this an ops agent rather than a doc-QA bot.

**M5 - LangGraph multi-agent loop with HITL.** Router/planner node plus tool-calling nodes, with a human-in-the-loop approval checkpoint (concrete placement decided at this milestone - e.g. before asserting a deprecation/breaking-change claim, or before spending GitHub API quota on an ambiguous issue search).

**M6 - Deep-agent-style planning.** Planner decomposes multi-part questions (e.g. "what changed between 0.100 and 0.115, and is issue #X about it still open?") into sub-tasks delegated to specialized sub-agents (doc retrieval, changelog, issue search), then synthesizes.

**M7 - Eval harness.** Golden Q&A set (~30-50 real FastAPI support questions with expected answers/citations), LLM-as-judge scoring (correctness, groundedness, citation accuracy), regression tracking across changes.

**M8 - Guardrails.** Block off-topic/unsafe queries; defend against prompt injection carried in retrieved GitHub issue text (a realistic injection vector, since issues are user-generated, unlike the curated docs).

**M9 - Phase 7: Production.** Dockerize, deploy behind a simple API on Render or Fly.io (platform choice deferred to this milestone), LangSmith tracing throughout, basic latency/cost/error-rate monitoring, least-privilege tool design (e.g. read-only-scoped GitHub token), and a kill switch.

**M10 - Phase 8: Portfolio.** Real README (what it does, why, how to run it, eval results) and a short write-up (problem/approach/trade-offs/results) for interview/client use.

## 6. How we work together

- One concept taught fully, then stop and wait for "next" - never bundle multiple concepts in one go.
- First appearance of any new library/tool/framework gets a standalone what-is-it/why-are-we- using-it explanation before any code.
- "Move fast" means execute quickly once a plan is agreed - never means skip the what/why.
- Don't rely on training-data recall for SDK/API specifics, especially anything Phase-7- related (Docker, LangSmith, Render/Fly.io) - verify against current docs/web since this shifts often.
- Recap the previous milestone's takeaways before starting the next one.

## 7. Verification approach per milestone

- M1-M3: run ingestion/retrieval scripts against the live FastAPI docs corpus and inspect chunk/citation quality directly.
  - M4-M6: interactively drive the LangGraph app with real multi-part questions, confirm tool calls, HITL interrupts, and sub-agent delegation fire as expected.
  - M7: run the eval harness and inspect judge scores against the golden set; re-run after later milestones to confirm no regression.
  - M8: manually attempt off-topic and prompt-injection queries to confirm guardrails hold.
  - M9: docker build/run locally, hit the deployed endpoint, confirm LangSmith traces appear and the kill switch works.
  - M10: fresh-eyes read of the README/write-up as if seeing it cold as an interviewer.
-

*Generated as a standing reference for the capstone project so scope and decisions are not lost across sessions. Source of truth for edits going forward: the plan file and this project's own README once M10 exists.*